

Implementasi dan Analisis Arsitektur Algoritma Pengurutan Data (Bubble Sort) pada Memori Internal dan Eksternal Menggunakan IP Core Oregano 8051

¹M. Altof, ²Rizqi Ageng Rinando, ³Muhammad Ridwan, ⁴Azka Tunggul Widita, ⁵Eldy Biru Samudera
¹[24/542271/PA/23021], ²[24/545013/PA/23149], ³[24/543440/PA/23087], ⁴[24/543536/PA/23093], ⁵[24/545239/PA/23165]

Department of Computer Science and Electronics

Universitas Gadjah Mada

Yogyakarta, Indonesia

KELAS ELA

¹maltof2006@mail.ugm.ac.id, ²rizqiagengrinando@mail.ugm.ac.id, ³muhammadridwan2005@mail.ugm.ac.id

⁴azkatunggulwidita@mail.ugm.ac.id, ⁵eldybirusamudera@mail.ugm.ac.id

Abstract—Proyek ini membahas simulasi dan analisis arsitektur mikrokontroler 8051 menggunakan platform IP Core Oregano 8051 berbasis VHDL. Eksperimen difokuskan pada implementasi algoritma pengurutan data (Bubble Sort) pada tingkat perangkat keras untuk mengevaluasi kinerja jalur data (datapath) dan manajemen memori. Simulasi dilakukan menggunakan perangkat lunak ModelSim Intel FPGA Starter Edition 2020.1 untuk mengamati perilaku organisasi komputer secara langsung pada tingkat sinyal (waveform). Program dieksekusi dalam dua skenario utama: pengurutan data pada General Purpose RAM (GPRAM) internal dan modifikasi tingkat lanjut menggunakan memori eksternal (RAMX) melalui instruksi khusus MOVX. Hasil eksperimen menunjukkan bahwa instruksi berhasil dikompilasi tanpa galat, eksekusi berjalan stabil hingga mencapai infinite loop sebagai mekanisme halt state, dan tabel memori mengonfirmasi keberhasilan penukaran alamat secara sekuensial dari kondisi acak [05H, 02H, 09H, 01H, 03H] menjadi terurut [01H, 02H, 03H, 05H, 09H]. Melalui pendekatan Register Transfer Level (RTL), dianalisis pula mekanisme instruction fetch, operasi Arithmetic Logic Unit (ALU), pengelolaan pointer, serta perbedaan pemetaan bus data pada struktur memori internal dan eksternal.

Index Terms—Oregano 8051, ModelSim, VHDL, Organisasi Komputer, Arsitektur Komputer, Bubble Sort, Memori Eksternal.

I. PENDAHULUAN

Organisasi dan arsitektur komputer merupakan disiplin ilmu yang menjembatani logika pemrograman perangkat lunak tingkat tinggi dengan struktur fisik perangkat keras yang mengeksekusinya. Dalam memahami fungsionalitas mikroprosesor atau mikrokontroler, konsep-konsep seperti siklus instruksi (instruction cycle), peran Arithmetic Logic Unit (ALU), alokasi memori, penanganan instruksi percabangan, serta kendali pewaktuan (clock timing) sering kali menjadi sangat abstrak jika hanya dipelajari melalui pemodelan teoretis. Untuk menjembatani kesenjangan pemahaman tersebut, diperlukan suatu metode simulasi di tingkat Register Transfer

Level (RTL). Simulasi ini mampu memvisualisasikan pergerakan data antar-register, interaksi bus, dan aktivasi sinyal kendali di setiap detak pewaktu secara real-time.

Proyek ini memanfaatkan Oregano 8051, sebuah intellectual property (IP) core komersial bersumber terbuka yang ditulis menggunakan bahasa deskripsi perangkat keras VHDL (Very High Speed Integrated Circuit Hardware Description Language). IP Core ini mengimplementasikan set instruksi (Instruction Set Architecture / ISA) standar mikrokontroler MCS-51 secara penuh. Platform Oregano 8051 menjadi objek studi yang ideal untuk analisis arsitektur komputer tingkat sarjana karena tetap mempertahankan kesederhanaan arsitektur 8-bit yang mudah dipahami, namun memiliki kelengkapan komponen fundamental dari sistem komputer modern. Komponen-komponen tersebut meliputi Program Counter (PC), Akumulator (Register A), Register B, Stack Pointer (SP), Data Pointer (DPTR), Program Status Word (PSW), serta antarmuka bus memori yang terpisah antara program (ROM) dan data (RAM).

Tujuan utama dari eksperimen ini adalah mendesain, menjalankan, dan menganalisis program tingkat rendah (Assembly) yang mengimplementasikan algoritma pengurutan data (Bubble Sort). Pemilihan algoritma pengurutan ini didasarkan pada karakteristiknya yang secara intensif menguji jalur data (datapath) dari CPU. Algoritma Bubble Sort menuntut prosesor untuk melakukan serangkaian operasi arsitektural yang kompleks secara berulang, meliputi:

- 1) Proses pembacaan memori (load) untuk mengambil data dari RAM.
- 2) Operasi perbandingan (compare) nilai melalui pengurangan di dalam ALU.
- 3) Evaluasi flag status pada register kontrol untuk menentukan kondisi pertukaran.
- 4) Modifikasi penunjuk alamat (pointer increment/decrement) pada register indeks.

- 5) Pencabangan bersyarat (conditional branching) dan tidak bersyarat (unconditional jump).
- 6) Penulisan kembali ke memori (store) untuk memperbarui posisi data yang telah ditukar.

Selain implementasi dasar pada memori internal, laporan ini juga menyertakan eksperimen modifikasi tingkat lanjut untuk mengeksekusi algoritma pada ruang memori eksternal prosesor (RAMX). Pendekatan komparatif ini bertujuan untuk memberikan analisis yang lebih komprehensif mengenai mekanisme manajemen bus data, bus alamat, dan sinyal kendali eksternal pada arsitektur Harvard yang dimodifikasi milik 8051. Dengan demikian, perbedaan efisiensi siklus mesin antara akses memori internal dan eksternal dapat diukur dan dievaluasi secara struktural.

II. PLATFORM OREGANO 8051 DAN LINGKUNGAN SIMULASI

Oregano 8051 berbeda dengan aplikasi simulator visual konvensional berbasis perangkat lunak (seperti EdSim51 atau MCU8051IDE) yang hanya mengemulasikan hasil akhir dari eksekusi instruksi. Oregano 8051 merupakan representasi dari deskripsi perangkat keras fisik (hardware description) yang dirancang untuk disintesis ke dalam silikon chip seperti FPGA (Field Programmable Gate Array) atau ASIC (Application-Specific Integrated Circuit). Oleh karena itu, pengujian jalannya program tidak dapat dilakukan melalui eksekusi interpreter biasa, melainkan harus melalui proses kompilasi kode VHDL dan simulasi perilakunya menggunakan perangkat lunak simulator sintesis tingkat industri, yaitu ModelSim Intel FPGA Starter Edition.

A. Arsitektur Modul Oregano 8051

Berdasarkan tinjauan arsitektur komputer, struktur kode VHDL pada Oregano 8051 mendemonstrasikan prinsip modularitas yang sangat tinggi, di mana unit-unit fungsional prosesor dipisahkan ke dalam beberapa modul entitas yang saling terhubung melalui bus internal. Struktur modular tersebut dijabarkan sebagai berikut:

- **Core dan Control Unit (mc8051_core.vhd):** Bagian inti dari prosesor yang bertindak sebagai otak dari sistem. Komponen ini bertanggung jawab untuk mengatur siklus hidup instruksi (fetch, decode, execute). Control Unit mengambil opcode dari memori program, menerjemahkannya melalui logika kombinasional internal, dan kemudian membangkitkan deretan sinyal kendali (control signals) yang didistribusikan ke seluruh register, ALU, dan antarmuka memori untuk mengoordinasikan operasi sistem.
- **ALU (Arithmetic Logic Unit — mc8051_alu.vhd):** Modul independen yang dirancang untuk melakukan operasi aritmatika 8-bit (penjumlahan, pengurangan, perkalian, pembagian) dan operasi logika (AND, OR, XOR, NOT, rotation). ALU menerima masukan utama dari Akumulator (Register A) dan register pembantu (seperti Register B atau internal buffer). Output dari ALU tidak hanya menghasilkan data hasil operasi, tetapi juga

memperbarui bit-bit pada Program Status Word (PSW), terutama Carry Flag (*CY*), Auxiliary Carry (*AC*), Overflow Flag (*OV*), dan Parity Bit (*P*).

- **ROM Interface:** Modul antarmuka memori program yang bertugas mengirimkan alamat instruksi 16-bit yang tersimpan di dalam Program Counter (melalui port keluaran `rom_addr_o`) ke memori instruksi eksternal, dan menerima kembali deretan data instruksi 8-bit (melalui port masukan `rom_data_i`) pada siklus detak berikutnya.
- **RAM Interface:** Modul kendali penyimpanan internal yang mengelola akses siklus cepat (fast-cycle access) menuju 256 byte RAM internal. Ruang memori ini terbagi menjadi 32 byte untuk bank register (Register Bank 0-3), 16 byte memori yang dapat dialamatkan per bit (bit-addressable RAM), dan 80 byte General Purpose RAM (GPRAM), serta Special Function Registers (SFR) pada alamat atas.
- **RAMX Interface:** Antarmuka khusus yang dirancang untuk menjembatani CPU dengan ruang memori data eksternal yang dapat dikembangkan hingga kapasitas 64 Kilobyte. Jalur bus data dan bus alamat pada antarmuka ini bersifat pasif selama operasi normal dan hanya akan diaktifkan secara eksklusif oleh Unit Kontrol ketika prosesor mendeteksi dan mengeksekusi instruksi khusus MOVX (Move External).

B. Integrasi Testbench

Dalam pengujian berbasis deskripsi perangkat keras, entitas fisik dari prosesor Oregano 8051 diisolasi dan dimasukkan (instantiated) ke dalam sebuah modul uji eksternal yang disebut sebagai testbench (`tb_mc8051_top_sim.vhd`). Modul testbench ini tidak disintesis menjadi perangkat keras, melainkan berfungsi untuk mereplikasi kondisi lingkungan laboratorium fisik yang ideal bagi chip prosesor.

Testbench bertindak sebagai osilator kristal eksternal yang menyuplai sinyal detak (clock) secara periodik dan kontinu dengan parameter waktu yang dapat diatur (misalnya periode detak 20 ns untuk mensimulasikan frekuensi kerja 50 MHz). Selain itu, testbench mengendalikan sinyal reset asinkron pada awal simulasi. Sinyal reset ini dipaksa berada pada kondisi aktif (logik '1') selama beberapa siklus awal untuk mengatur seluruh muatan register internal, mengosongkan jalur pipa instruksi, dan memaksa nilai Program Counter (PC) kembali ke alamat inisialisasi awal 0000H. Setelah kondisi awal yang valid tercapai, sinyal reset diturunkan menjadi logik '0', sehingga prosesor dapat mulai melakukan fetch instruksi pertama dari memori ROM yang juga telah diintegrasikan di dalam komponen testbench.

III. DESAIN DAN LOGIKA PROGRAM

Program yang dirancang dalam eksperimen ini mengimplementasikan algoritma Bubble Sort, sebuah metode pengurutan data konvensional yang bekerja dengan cara membandingkan dua elemen data yang berdekatan secara berurutan, dan menukarnya jika urutannya salah. Proses ini diulang dari awal

hingga seluruh data berada pada posisi yang benar. Kode sumber ditulis dalam bahasa tingkat rendah, yaitu Assembly MCS-51, guna memastikan kontrol mutlak atas pemanfaatan register internal dan minimasi penggunaan instruksi demi efisiensi eksekusi arsitektural. Kompilasi kode dilakukan menggunakan assembler industri Keil μ Vision untuk menghasilkan berkas biner objek.

A. Skenario Utama (Memori Internal)

Pada skenario pertama, algoritma Bubble Sort diimplementasikan untuk mengurutkan lima buah data acak berukuran 8-bit yang disimpan pada memori internal General Purpose RAM (GPRAM). Data acak yang digunakan adalah 05H, 02H, 09H, 01H, dan 03H. Data ini dialokasikan secara berurutan pada alamat blok memori internal mulai dari 40H hingga 44H.

Secara arsitektural, program memanfaatkan dua teknik pengalamatan utama yang disediakan oleh set instruksi MCS-51:

- 1) **Immediate Addressing Mode:** Teknik ini digunakan untuk memasukkan nilai konstanta secara langsung ke dalam register tanpa perlu membaca dari alamat memori data. Mode ini diimplementasikan pada tahap inisialisasi awal nilai data acak ke dalam memori (misalnya, MOV 40H, #05H), serta untuk mengatur batas perulangan (loop counter) pada register pembantu (misalnya, MOV R6, #04H).
- 2) **Indirect Addressing Mode:** Teknik ini merupakan kunci utama dalam implementasi algoritma yang melibatkan array atau urutan data. Dengan menggunakan tanda ampersand (@), register R0 dan R1 dialokasikan sebagai pointer (penunjuk alamat) dinamis. Sebagai contoh, instruksi MOV A, @R0 berarti CPU akan membaca data dari alamat RAM yang nilainya sedang disimpan di dalam register R0, bukan membaca nilai dari register R0 itu sendiri. Hal ini memungkinkan pemindahan penunjuk alamat dilakukan hanya dengan menaikkan nilai register indeks menggunakan instruksi INC R0.

Logika perbandingan nilai pada arsitektur 8051 dijalankan secara intensif oleh instruksi CJNE (Compare and Jump if Not Equal). Karena arsitektur 8051 tidak memiliki instruksi perbandingan murni yang berdiri sendiri tanpa mengubah kondisi register, instruksi CJNE bekerja dengan cara membandingkan dua operan melalui sirkuit pengurangan di dalam ALU. Nilai elemen pertama dimuat ke dalam Akumulator, kemudian dibandingkan dengan elemen kedua. Jika kedua nilai tidak sama, program akan melompat ke label tujuan.

Secara bersamaan, operasi pengurangan di ALU tersebut memengaruhi kondisi Carry Flag (CY) pada register PSW. Jika nilai operan pertama (di sebelah kiri) lebih kecil dari operan kedua (di sebelah kanan), maka operasi pengurangan akan menghasilkan kondisi borrow (pinjam), yang menyebabkan Carry Flag diset menjadi '1'. Sebaliknya, jika operan pertama lebih besar, Carry Flag akan bernilai '0'. Berdasarkan status bit Carry Flag ini, instruksi pencabangan bersyarat JNC (Jump if No Carry) digunakan untuk menentukan apakah mekanisme pertukaran data (swap) perlu dieksekusi atau dilewati. Jika

data sebelah kiri lebih besar, pertukaran dilakukan dengan memanfaatkan Register B sebagai wadah penyimpanan sementara (temporary register) agar tidak merusak data asli selama proses transfer antarsel RAM.

B. Modifikasi Eksperimen: Eksekusi pada Memori Eksternal (RAMX)

Untuk menguji dan menganalisis karakteristik arsitektur bus data dan bus alamat eksternal pada Oregon 8051, program dimodifikasi agar beroperasi sepenuhnya pada ruang memori eksternal (RAMX). Data acak dengan komposisi yang sama dialokasikan pada alamat eksternal mulai dari 0040H.

Perubahan mendasar yang terjadi pada modifikasi ini terletak pada instruksi akses memori yang digunakan. Instruksi standar MOV secara arsitektural tidak memiliki jalur sirkuit untuk merutekan sinyal keluar dari batas fisik chip inti menuju pin eksternal. Oleh karena itu, seluruh instruksi pembacaan dan penulisan memori harus digantikan dengan instruksi khusus MOVX (Move External).

Penggunaan instruksi MOVX membawa implikasi arsitektural yang sangat signifikan terhadap efisiensi jalur data (datapath). Pada arsitektur MCS-51, instruksi MOVX menuntut penggunaan Akumulator (Register A) secara eksklusif sebagai satu-satunya gerbang atau jembatan untuk mentransfer data dari dan ke luar chip. Selain itu, penunjukan alamat 16-bit untuk memori eksternal harus dikelola oleh register Data Pointer (DPTR) atau menggunakan register R0/R1 untuk pengalamatan eksternal 8-bit halaman rendah.

Keterbatasan struktural ini menciptakan fenomena ketergantungan data (data dependency) yang sangat tinggi terhadap Akumulator. Proses pertukaran data (swap) yang pada memori internal dapat dilakukan secara fleksibel, pada memori eksternal harus disusun melalui serangkaian instruksi sekuensial yang ketat, di mana Akumulator harus dikosongkan dan diisi kembali secara bergantian. Kode implementasi modifikasi untuk loop dalam pengurutan memori eksternal disajikan pada Gambar 1.

```

28 LOOP_DALAM:
29     MOV A, R0
30     ADD A, #01H
31     MOV R1, A
32     MOVX A, @R0
33     MOV B, A
34     MOVX A, @R1
35
36     CJNE A, B, CEK_BESAR
37     JMP LANJUT
38 CEK_BESAR:
39     JNC LANJUT
40     MOVX @R0, A
41     MOV A, B
42     MOVX @R1, A
43 LANJUT:
44     INC R0
45     DJNZ R7, LOOP_DALAM ; geser pointer
46     DJNZ R6, LOOP_LUAR
47 SELESAI:
48     SJMP SELESAI ; halt program
49 END

```

Fig. 1. Cuplikan modifikasi algoritma Bubble Sort menggunakan pengalaman MOVX untuk memori eksternal.

IV. METODOLOGI EKSPERIMEN

Siklus pengembangan dan pengujian perangkat keras dalam eksperimen ini diatur melalui metodologi toolchain yang terkendali dengan ketat. Hal ini penting untuk memastikan bahwa kode program Assembly dapat diterjemahkan menjadi representasi memori biner yang kompatibel dengan bahasa VHDL Oregano 8051 tanpa menimbulkan distorsi instruksi. Tahapan metodologi tersebut dijabarkan sebagai berikut:

- 1) **Kompilasi Jalur Perakitan (Assembly Compilation):** Kode sumber Assembly yang telah ditulis (.asm) dimasukkan ke dalam lingkungan kerja Keil μ Vision. Proses kompilasi dilakukan dengan target arsitektur standar 8051. Proses *Build* harus dipastikan menghasilkan status 0 Error(s), 0 Warning(s). Keberhasilan tahap ini menjamin bahwa seluruh sintaks kode telah mematuhi batasan Opcode resmi dari ISA MCS-51. Hasil akhir dari tahapan ini adalah berkas berformat Intel HEX.
- 2) **Translasi Format Memori (Hex to DUA Conversion):** Berkas Intel HEX hasil kompilasi tidak dapat dibaca secara langsung oleh simulator VHDL ModelSim karena perbedaan representasi data. Untuk mengatasi hal ini, digunakan sebuah skrip utilitas berbasis Python yang bernama *converter.py*. Skrip ini memetakan setiap baris alamat dan data dari format Intel HEX, kemudian menformulasikannya kembali menjadi format berkas biner internal yang bernama DUA (*mc8051_rom.dua*). Berkas ini bertindak sebagai isi muatan memori ROM awal yang akan dibaca oleh modul VHDL Oregano saat simulasi dimulai.
- 3) **Preparasi Lingkungan Simulasi:** Perangkat lunak ModelSim Intel FPGA Starter Edition diluncurkan melalui eksekusi dengan hak akses administrator sistem (*Run as Administrator*). Kebijakan perizinan tingkat tinggi ini bersifat esensial dalam simulasi perangkat keras lokal. Hal ini dikarenakan ModelSim memerlukan izin penulisan direktori secara simultan untuk membuat struktur pustaka kerja internal (*vlib work*) serta mengompilasi seluruh berkas dependensi kode VHDL Oregano (seperti *mc8051_p0.vhd*, *mc8051_alu.vhd*, dan *mc8051_core.vhd*) ke dalam repositori lokal komputer tanpa terblokir oleh sistem keamanan operasi.
- 4) **Eksekusi Otomatisasi Testbench:** Melalui konsol komando (Transcript) pada ModelSim, proses eksekusi dijalankan menggunakan perintah makro berbasis skrip .do. Langkah pertama adalah mengeksekusi *do mc8051_compile.do* untuk melakukan kompilasi menyeluruh terhadap sirkuit logika VHDL. Setelah struktur interkoneksi selesai dibangun, perintah *do mc8051_sim.do* dijalankan untuk memicu testbench, mengaktifkan osilator clock, memasukkan sinyal reset, dan merender seluruh aktivitas gelombang elektrik (waveform) ke dalam jendela Wave Default hingga batasan waktu simulasi terpenuhi.

V. HASIL EKSEKUSI DAN ANALISIS ORGANISASI KOMPUTER

Analisis hasil eksekusi dibagi menjadi tiga parameter evaluasi utama: analisis pergerakan sinyal bus (datapath), analisis kondisi akhir penguncian prosesor (halt state), dan verifikasi mutasi data pada matriks memori RAM internal. Untuk membuktikan konsep Register Transfer Level (RTL), fungsionalitas komponen IP Core Oregano 8051 dipetakan terhadap logika algoritma Bubble Sort yang dieksekusi. Detail pemetaan interaksi perangkat keras selama simulasi dirangkum pada Tabel I.

TABLE I
PEMETAAN KOMPONEN RTL TERHADAP EKSEKUSI BUBBLE SORT

Komponen VHDL	Port/Sinyal Utama	Peran dalam Eksperimen Bubble Sort
mc8051_core	rom_addr_o	Mengirimkan alamat instruksi secara inkremental dan mengelola pencabangan looping.
mc8051_alu	acc_i, cy_i	Mengeksekusi pengurangan untuk instruksi CJNE guna menentukan data terbesar/terkecil.
mc8051_ram	ram_data_i/o	Menyimpan dan memperbarui urutan array memori internal pada alamat 40H–44H.
mc8051_ramx	datax_i/o, wrx_o	Merutekan data ke luar chip saat instruksi MOVX tereksekusi pada eksperimen modifikasi.
Register PSW	Carry (CY) Flag	Bertindak sebagai indikator penentu bagi instruksi JNC untuk melakukan proses pertukaran data (swap).

A. Analisis Sinyal Waveform (Datapath)

Aktivitas internal sirkuit logik prosesor selama proses pengambilan dan eksekusi instruksi dapat diamati secara mendalam melalui jendela Waveform ModelSim (terbaca sebagai representasi grafis dari Gambar 2 pada laporan fisik).

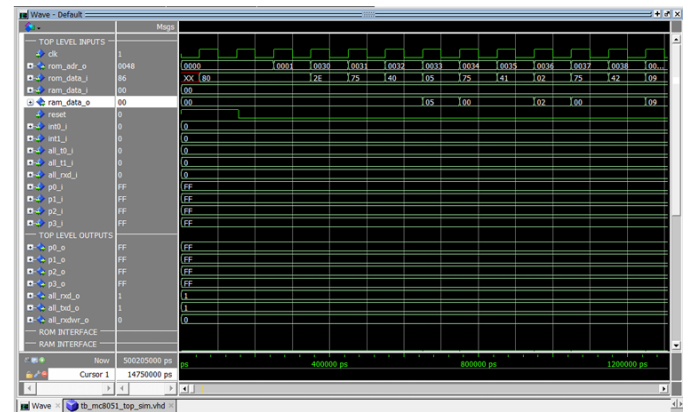


Fig. 2. Rekaman aktivitas sinyal pada bus sistem saat tahap inisialisasi dan pengambilan instruksi (fetch).

Siklus hidup instruksi (instruction cycle) tervisualisasi secara akurat pada transisi pulsa clock. Ketika sinyal reset diturunkan dari nilai '1' ke '0', Program Counter (PC) menginisialisasi alamat awal di titik 0000H. Pada jendela

sinyal, port `rom_addr_o` mengeluarkan nilai `0000H`. Sebagai respons, memori ROM mengirimkan biner opcode `80H` melalui bus `rom_data_i` pada tepi naik clock berikutnya. Unit kontrol mengidentifikasi `80H` sebagai instruksi `SJMP` (Short Jump) yang merupakan instruksi pencabangan tidak bersyarat.

Arsitektur organisasi komputer terverifikasi ketika nilai PC melompat secara instan dari `0001H` langsung menuju alamat `0030H`, melewati area memori bawah. Hal ini membuktikan bahwa Unit Kontrol telah berhasil mengeksekusi operasi penambahan nilai offset pencabangan (`2EH`) ke dalam register PC tanpa mengalami distorsi logika atau delay struktural.

Setelah berada di alamat `0030H`, bus `rom_data_i` menangkap opcode `75H`. Berdasarkan arsitektur set instruksi `8051`, `75H` diidentifikasi sebagai instruksi `MOV direct, #data` yang membutuhkan 3 byte memori (3 siklus fetch). Unit kontrol secara sekuensial mengambil byte kedua yaitu `40H` (sebagai alamat RAM internal tujuan) dan byte ketiga yaitu `05H` (sebagai data murni). Efek dari eksekusi ini terlihat jelas pada bus internal `ram_data_o` yang sesaat kemudian melewati nilai `05H` untuk ditulis ke dalam matriks RAM internal. Proses ini mengonfirmasi bahwa jalur data (datapath) interkoneksi antara Unit Kontrol, register instruksi, dan memori data telah berfungsi dengan determinisme yang absolut.

B. Kondisi Akhir Program (Halt State / Idle State)

Dalam arsitektur komputer berbasis mikrokontroler, sebuah program tidak diperbolehkan menyelesaikan eksekusi hingga keluar dari ruang memori program (program counter overrun). Jika PC terus bertambah nilainya melewati batas akhir kode yang ditulis, CPU akan membaca data acak (garbage data) atau opcode kosong (`00H` - `NOP`) pada sirkuit ROM, yang dapat menyebabkan perilaku sistem menjadi tidak dapat diprediksi (undetermined behavior).

Untuk mencegah malfungsi tersebut, kode program Bubble Sort ditutup dengan instruksi `SJMP SELESAI` (atau `SJMP $` dalam notasi perakitan). Berdasarkan pengamatan gelombang waveform pada titik akhir waktu simulasi (rentang parameter mendekati `500.205 ns`), nilai pada port bus `rom_addr_o` terkunci dan hanya berfluktuasi secara kontinu di antara dua alamat biner yang sama secara berulang-ulang. Prosesor terperangkap di dalam putaran looping sempit yang disengaja. Kondisi ini secara organisasi komputer disebut sebagai `Halt State` atau `Idle State`. Mekanisme ini memastikan keselamatan sistem dengan mengunci pergerakan PC, sehingga menjaga kestabilan data yang telah selesai diproses di dalam RAM dan melindungi prosesor dari kegagalan akibat luapan instruksi.

C. Bukti Pengurutan Memori (Internal RAM)

Verifikasi fungsionalitas algoritma secara mutlak dibuktikan melalui pembedahan status internal sel memori menggunakan fitur Memory List View pada ModelSim. Dengan menavigasi ke jalur hirarki sirkuit RAM internal, yaitu `/i_mc8051_top/i_mc8051_ram/gpram`, dan mengubah basis bilangan (radix) tampilan menjadi heksades-

imal, hasil akhir pengurutan data dapat diekstraksi secara visual.

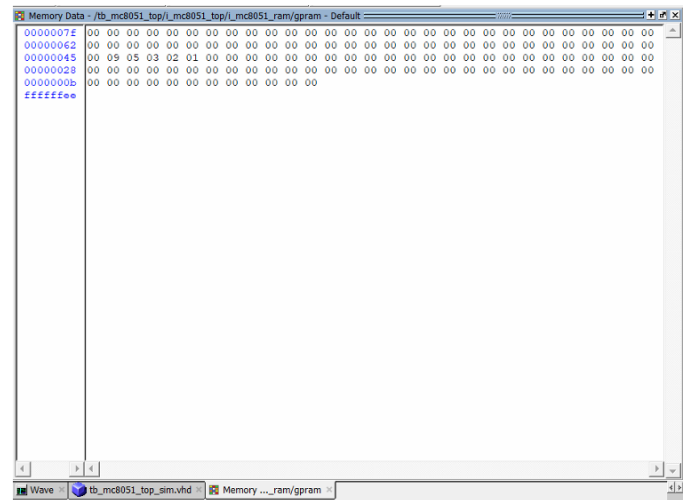


Fig. 3. Pemetaan nilai heksadesimal pada gpram internal pasca-eksekusi algoritma Bubble Sort.

Sebelum program dijalankan, sel alamat `40H` hingga `44H` berisi data acak berturut-turut: `05H`, `02H`, `09H`, `01H`, dan `03H`. Setelah simulasi mencapai kondisi `halt`, inspeksi visual menunjukkan bahwa komposisi data pada area sel tersebut telah termutasi sepenuhnya. Matriks memori menampilkan urutan biner baru: alamat `40H` berisi `01H`, `41H` berisi `02H`, `42H` berisi `03H`, `43H` berisi `05H`, dan `44H` berisi `09H`. Data telah terurut secara naik (ascending) dengan sempurna. Hasil ini membuktikan secara empiris bahwa seluruh logika pencabangan bersyarat, operasi perbandingan pengurangan pada ALU, penanganan bit tanda pada Carry Flag, serta manipulasi indeks alamat menggunakan mode pengalamatan tidak langsung (`@R0`) telah bekerja secara sinkron tanpa kesalahan fungsional satu bit pun.

VI. MODIFIKASI TINGKAT LANJUT: AKSES RAM EKSTERNAL

Guna meninjau kemampuan bus sistem secara menyeluruh, simulasi diputar ulang dengan parameter pointer yang diarahkan pada perangkat memori di luar matriks sirkuit internal. Keterbatasan fisik yang dieksplorasi dalam rekonfigurasi ini adalah absennya sirkuit langsung antara memori RAMX dengan register selain Akumulator. Pada kode yang ditunjukkan di Gambar 1, swap mekanis dialihkan sepenuhnya agar selalu mengalir (routed) melalui bus Akumulator.

Melalui inspeksi sub-komponen perangkat keras pada jalur hirarki `/i_mc8051_top/i_mc8051_ramx/p_readwrite/gpram`, dikonfirmasi bahwa sirkuit luar chip telah aktif. Data biner pada memori eksternal berhasil diurutkan menjadi `01H`, `02H`, `03H`, `05H`, dan `09H`. Eksperimen ini berhasil memverifikasi kemampuan IP Core Oregano 8051 dalam mereplikasi demarkasi alamat memori yang merupakan parameter kritis

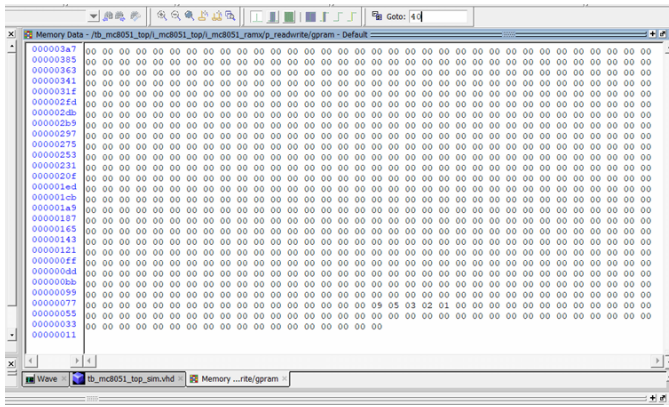


Fig. 4. Verifikasi pengurutan data sekuensial pada modul memori eksternal (RAMX).

spesifikasi Von Neumann Architecture yang dimodifikasi milik MCS-51.

Namun, dari sudut pandang arsitektur komputer dan efisiensi performa, terdapat perbedaan performa yang sangat signifikan antara eksekusi memori internal (skenario A) dan memori eksternal (skenario B). Perbandingan mendalam tersebut dianalisis melalui aspek-aspek struktural berikut:

1) *Bottleneck pada Struktur Jalur Data (Datapath Bottleneck)*: Pada memori internal, CPU dapat melakukan operasi transfer data secara langsung antar-register atau dari register menuju sel memori menggunakan instruksi MOV yang fleksibel. Namun, pada memori eksternal, instruksi MOVX mewajibkan seluruh aliran data masuk (read) dan data keluar (write) disalurkan melalui satu-satunya register tunggal, yaitu Akumulator (Register A).

Fenomena ini menyebabkan terjadinya penyumbatan jalur data (accumulator choking). Register B atau register serbaguna lainnya (R0-R7) kehilangan kemampuan untuk menerima data secara langsung dari memori luar. Akibatnya, untuk melakukan pertukaran (swap) dua data pada memori eksternal, program membutuhkan jumlah operasi pemindahan antara memori-ke-register yang jauh lebih panjang, yang meningkatkan dependensi data internal.

2) *Kompleksitas Siklus Instruksi dan Konsumsi Waktu (Clock Cycle Overhead)*: Secara arsitektural, setiap instruksi memiliki bobot waktu eksekusi yang diukur dalam satuan Siklus Mesin (Machine Cycle). Pada arsitektur standar 8051, satu siklus mesin setara dengan 12 detak osilator clock.

- Instruksi pembacaan memori internal seperti MOV A, @R0 hanya membutuhkan 1 Siklus Mesin karena sirkuit memori berada dalam satu cetakan silikon dekat dengan Unit Kontrol.
- Sebaliknya, instruksi akses memori eksternal MOVX A, @R0 membutuhkan minimal 2 Siklus Mesin.

Satu siklus tambahan (overhead cycle) secara fisik diperlukan oleh Unit Kontrol untuk mengaktifkan sirkuit latch alamat eksternal, membuka gerbang tri-state bus, dan memberikan waktu tunggu (wait state) bagi memori luar untuk

menstabilkan level tegangan biner pada pin data eksternal sebelum dibaca oleh prosesor.

3) *Analisis Kuantitatif Efisiensi Kode*: Secara organisasi komputer, efisiensi eksekusi algoritma Bubble Sort dapat diukur dari konsumsi siklus mesin dan dependensi struktural. Tabel II berikut merangkum perbandingan metrik performa arsitektural antara penggunaan memori internal dan memori eksternal.

TABLE II
PERBANDINGAN METRIK PERFORMA ARSITEKTURAL INTERNAL VS. EKSTERNAL

Parameter	Arsitektur	Memori Internal (GPRAM)	Memori Eksternal (RAMX)
Instruksi Utama	Akses	MOV A, @Ri / MOV @Ri, A	MOVX A, @DPTR / MOVX @Ri, A
Kebutuhan Mesin (Fetch-Execute)	Siklus (Fetch-Execute)	1 Siklus (Cepat)	2 Siklus (Terkena Wait State)
Jalur Data	Routing Bus	Bus internal langsung (satu die silikon)	Melalui Port Eksternal & Sirkuit Latch
Dependensi (Bottleneck)	Akumulator	Rendah (Bisa menggunakan Register B & R0-R7)	Mutlak (Seluruh I/O wajib melewati Akumulator)
Efisiensi Swap Data	Mekanisme	Sangat Tinggi	Rendah (Overhead instruksi pemindahan tinggi)

Secara organisasi komputer, hal ini menunjukkan bahwa penggunaan memori eksternal pada arsitektur 8051 memberikan keuntungan dalam hal ekspansi kapasitas ruang penyimpanan (hingga 64 KB), namun harus dibayar mahal dengan penurunan efisiensi kecepatan eksekusi komputasi (performance degradation) yang mencapai lebih dari 100% dibandingkan dengan pemanfaatan register internal.

VII. KESIMPULAN

Proyek simulasi arsitektur IP Core Oregano 8051 yang diintegrasikan dengan perangkat lunak ModelSim Intel FPGA Starter Edition telah terbukti menjadi fasilitas laboratorium virtual yang solid dan akurat untuk mengkaji parameter perangkat keras tingkat rendah (low-level hardware). Program algoritma Bubble Sort yang dikomposisikan dari fondasi bahasa Assembly MCS-51 dan direalisasikan pada dua spektrum arsitektur memori yang berbeda (RAM Internal konvensional dan eksternal RAMX) telah selesai disimulasikan secara sukses tanpa menunjukkan adanya distorsi fatal maupun kesalahan translasi pada sistem.

Ekstraksi dan pengkajian terhadap pergerakan register pointer, osilasi gelombang siklus clock, aktivitas mutasi pada bus alamat dan bus data, serta perubahan matriks tabel RAM pada kondisi akhir komputasi memberikan kepastian ilmiah bahwa setiap set instruksi secara konsisten diterjemahkan menjadi aksi elektrik perangkat keras dengan tingkat determinisme yang absolut. Melalui analisis komparatif antara modifikasi instruksi MOV dan MOVX, proyek ini berhasil menyoroti

perbedaan manajemen bus internal dan bus eksternal yang menjadi parameter krusial dalam pertimbangan optimasi performa desain sistem tertanam (embedded system).

REPOSITORY DOKUMENTASI

Berkas sumber sistem proyek (source code Assembly, skrip konfigurasi kompilasi, fail antarmuka DUA, dan tangkapan layar simulasi komprehensif) disandikan untuk kemudahan replikasi pengujian. Seluruh data tersebut diunggah dan dapat diakses pada repository publik GitHub berikut:

<https://github.com/altof03/orkom-oregano8051-bubblesort.git>

```
orkom-oregano8051-bubblesort/
+-- README.md
+-- src/
|   +-- bubble_sort_ext.asm
|   +-- bubble_sort_int.asm
|   +-- converter.py
+-- scripts/
|   +-- mc8051_compile.do
|   +-- mc8051_sim.do
|   +-- mc8051_wave.do
+-- results/
|   +-- Gambar screenshot Keil (bukti 0 Error).png
|   +-- Gambar screenshot Tabel RAM Internal (
|       bukti data terurut).png
|   +-- Gambar screenshot Tabel RAMX Eksternal (
|       bukti data terurut modifikasi).png
|   +-- Gambar screenshot Waveform (Kondisi Awal).
|       png
|   +-- regs.log
+-- report/
|   +-- Laporan Akhir.pdf
|   +-- Langkah Eksperimen.pdf
```

Listing 1. Struktur repository GitHub

REFERENCES

- [1] Oregano Systems, "MC8051 IP Core User Guide," *Documentation for MC8051 Design*, Austria.
- [2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann, 2020.
- [3] Intel Corporation, "MCS-51 Microcontroller Family User's Manual," 1994.
- [4] Mentor Graphics (Siemens), *ModelSim Intel FPGA Starter Edition User's Manual*, 2020.